

UNIVERSIDAD NACIONAL DE ENTRE RÍOS

Facultad de Ingeniería

Carrera: Licenciatura de Ciencia de Datos

Materia: Fundamentos de Algoritmos y Estructuras de Datos

Título del Trabajo: **Aplicaciones de estructuras jerárquicas y grafos**

Equipo del Docente:

Dr. Javier E. Diaz Zamboni

Bioing. Jordán F. Insfrán

Bioing. Diana Vertiz del Valle (de licencia)

Esp. Bioing. Juan Francisco Rizzato

Dr. Bioing. Feliciano Franco

Integrantes del equipo:

Franco Acevedo

Franco Tomiozzo

Comisión: 1

Fecha de entrega: 08/06/2026

Link del repositorio:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>



1. INTRODUCCIÓN

Kevin Kelvin es un científico que estudia el clima y necesita consultar frecuentemente temperaturas en un rango de fechas. El desafío está en que el conjunto de mediciones crece continuamente, por lo que se necesita una estructura que permita inserciones, búsquedas y consultas por rango de forma eficiente.

La solución implementada es una base de datos en memoria principal llamada `TemperaturasDB`, que utiliza internamente un Árbol AVL como estructura de almacenamiento. El AVL garantiza $O(\log n)$ para inserciones, búsquedas y eliminaciones, manteniéndose balanceado automáticamente. La clave de cada nodo es la fecha (objeto `datetime`), lo que permite ordenar las mediciones cronológicamente y realizar consultas por rango de forma eficiente mediante recorrido inorden acotado.

La implementación quedó dividida en tres capas: **nodoAVL.py** (nodo con fecha, temperatura y altura), **arbolAVL.py** (árbol AVL genérico con rotaciones y balanceo) y **Temperatura_DB.py** (interfaz de alto nivel que encapsula el árbol y expone los métodos pedidos por la consigna).

2. DESARROLLO DE LOS PROBLEMAS

A. Análisis y planteo

Para este problema se necesita una estructura que soporte eficientemente las siguientes operaciones: inserción ordenada por **fecha**, búsqueda por **fecha** exacta, eliminación por **fecha**, y consulta por rango de **fechas**. Se compararon las principales alternativas:

Estructura	Inserción	Búsqueda	Rango	¿Adecuada?
Lista desordenada	$O(1)$	$O(n)$	$O(n)$	No: búsqueda lenta
Lista ordenada	$O(n)$	$O(\log n)^*$	$O(k + \log n)$	No: inserción lenta
Árbol AVL	$O(\log n)$	$O(\log n)$	$O(k + \log n)$	Sí: es óptimo
Hash table	$O(1)$	$O(1)$	$O(n)$	No: rango ineficiente

* Búsqueda binaria en lista ordenada. **k = cantidad de elementos en el rango**.

Se eligió el Árbol AVL porque combina $O(\log n)$ para todas las operaciones individuales con $O(k + \log n)$ para consultas por rango, donde **k** es la cantidad de resultados. Ninguna otra estructura ofrece esta combinación. La hash table es más rápida para búsqueda exacta pero ineficiente para rangos, que es la operación más frecuente en el problema de Kevin.



B. Implementación / Resolución

Árbol AVL (estructura interna)

El **ArbolAVL** almacena nodos con clave **datetime** y valor **temperatura**. La propiedad **AVL** garantiza que la diferencia de alturas entre subárboles izquierdo y derecho de cualquier nodo es como máximo **1**. Cuando una inserción o eliminación rompe esta propiedad, se aplica la rotación correspondiente:

- **Rotación derecha (DD):** subárbol izquierdo más pesado.
- **Rotación izquierda (II):** subárbol derecho más pesado.
- **Rotación izquierda-derecha (ID):** hijo izquierdo con subárbol derecho más pesado.
- **Rotación derecha-izquierda (DI):** hijo derecho con subárbol izquierdo más pesado.

El método **_rebalancear** aplica la rotación correcta según el factor de balance del nodo y de su hijo:

```
def _rebalancear(self, nodo):
    self._actualizar_altura(nodo)
    b = self._factor_balance(nodo)
    if b > 1 and self._factor_balance(nodo.izquierdo) >= 0:    # II
        return self._rotar_derecha(nodo)
    if b > 1 and self._factor_balance(nodo.izquierdo) < 0:    # ID
        nodo.izquierdo = self._rotar_izquierda(nodo.izquierdo)
        return self._rotar_derecha(nodo)
    if b < -1 and self._factor_balance(nodo.derecho) <= 0:    # DD
        return self._rotar_izquierda(nodo)
    if b < -1 and self._factor_balance(nodo.derecho) > 0:    # DI
        nodo.derecho = self._rotar_derecha(nodo.derecho)
        return self._rotar_izquierda(nodo)
    return nodo    # ya balanceado, sin cambios
```

El método **_insertar** desciende recursivamente hasta la posición correcta y delega el rebalanceo al subir:

```
def _insertar(self, nodo, fecha, temperatura):
    if nodo is None:    # lugar vacío → crear nodo
        self._cantidad += 1
        return NodoAVL(fecha, temperatura)
    if fecha < nodo.fecha:
        nodo.izquierdo = self._insertar(nodo.izquierdo, fecha, temperatura)
    elif fecha > nodo.fecha:
        nodo.derecho = self._insertar(nodo.derecho, fecha, temperatura)
    else:    # fecha duplicada → actualizar temp
        nodo.temperatura = temperatura
        return nodo
    return self._rebalancear(nodo)
```



El método `_rango_inorden` recorre el árbol en orden cronológico acotado por `[f1, f2]`, lo que permite obtener los resultados ya ordenados sin necesidad de un sort posterior:

```
def _rango_inorden(self, nodo, f1, f2, resultado):
    if nodo is None: return
    if nodo.fecha > f1: self._rango_inorden(nodo.izquierdo, f1, f2, resultado) # poda izquierda
    if f1 <= nodo.fecha <= f2: resultado.append(nodo) # nodo en rango
    if nodo.fecha < f2: self._rango_inorden(nodo.derecho, f1, f2, resultado) # poda derecha
```

TemperaturasDB (interfaz de alto nivel)

TemperaturasDB encapsula el **ArbolAVL** y expone la interfaz definida por la consigna. Convierte las **fechas** en string a objetos **datetime** antes de delegarlas al árbol. Los métodos de rango obtienen la lista de nodos con `_rango_inorden` y luego extraen el valor pedido:

devolver_temperaturas retorna la lista ya ordenada cronológicamente (el recorrido inorden del AVL garantiza el orden), con el formato exacto pedido por la consigna:

```
def devolver_temperaturas(self, fecha1: str, fecha2: str) -> list:
    """
    Lista ordenada de mediciones en el rango. O(log n + k)

    Precondiciones:
    - fecha1 y fecha2 tienen formato dd/mm/aaaa
    - fecha1 <= fecha2

    Postcondición:
    - la lista retornada está ordenada cronológicamente (ascendente)
    - cada elemento tiene formato "dd/mm/aaaa: temperatura °C"
    """
    # -- precondiciones --
    self._validar_fecha_str(fecha1)
    self._validar_fecha_str(fecha2)
    f1, f2 = self._validar_rango(fecha1, fecha2)

    ns = self._arbol.rango(f1, f2)
    resultado = [f"{n.fecha.strftime(self.FORMATO_FECHA)}: {n.temperatura} °C" for n in ns]

    # -- postcondición --
    fechas_resultado = [n.fecha for n in ns]
    assert fechas_resultado == sorted(fechas_resultado), \
        "Postcondición fallida: la lista no está ordenada cronológicamente"

    return resultado
```

Carga desde archivo

cargar_desde_archivo: lee el archivo línea a línea, parsea cada registro con formato **dd/mm/aaaa;temperatura** e inserta cada medición en la base de datos. Las líneas con errores se ignoran con un aviso, sin interrumpir la carga:

```

def cargar_desde_archivo(self, ruta: str) -> int:
    """
    Precondición:
    - ruta es la ruta a un archivo de texto existente y legible.
    - Cada línea de datos tiene formato dd/mm/aaaa;temperatura.
    Postcondición:
    - Las líneas válidas son insertadas en la base de datos.
    - Las líneas con errores se ignoran y se imprime un aviso.
    - Retorna el número de mediciones cargadas exitosamente.
    """

    cargadas = errores = 0
    with open(ruta, "r", encoding="utf-8") as archivo:
        for num, linea in enumerate(archivo, start=1):
            linea = linea.strip()
            if not linea or linea.startswith("#"):
                continue
            partes = linea.split(";")
            try:
                if len(partes) != 2:
                    raise ValueError("Se esperaban 2 columnas: fecha;temperatura")
                fecha_str = partes[0].strip()
                temperatura = float(partes[1].strip())
                self.guardar_temperatura(temperatura, fecha_str)
                cargadas += 1
            except (ValueError, IndexError) as e:
                print(f"    Línea {num} ignorada: {e}")
                errores += 1
    print(f" Carga completada: {cargadas} mediciones cargadas, {errores} errores.")
    return cargadas

```

PRECONDICIONES Y POSTCONDICIONES

Todos los métodos de **ArbolAVL** y **TemperaturasDB** incluyen docstrings con pre y postcondiciones explícitas. La tabla resume los métodos principales:

Método	Precondición	Postcondición
guardar_temperatura (temp, fecha)	fecha es string dd/mm/aaaa ; temp es float.	Si fecha no existía, se agrega y cantidad aumenta en 1 . Si ya existía, temperatura queda actualizada.
devolver_temperatura (fecha)	fecha es string dd/mm/aaaa .	Retorna el float almacenado para esa fecha , o None si no existe.
max_temp_rango (f1, f2)	f1 <= f2 en orden cronológico; ambas son strings dd/mm/aaaa .	Retorna el float máximo en el rango [f1 , f2], o None si el rango está vacío.



min_temp_rango (f1, f2)	f1 <= f2 en orden cronológico.	Retorna el float mínimo en el rango [f1, f2], o None si el rango está vacío.
temp_extremos_rango (f1, f2)	f1 <= f2 en orden cronológico.	Retorna (min, max) del rango, o (None, None) si está vacío.
borrar_temperatura (fecha)	fecha es string dd/mm/aaaa .	Si fecha existía, la medición es eliminada y cantidad disminuye en 1.
devolver_temperaturas (f1, f2)	f1 <= f2 en orden cronológico.	Retorna lista de strings ' dd/mm/aaaa: T °C ' ordenada cronológicamente.
cantidad_muestras()	Ninguna.	Retorna entero >= 0 igual al número de mediciones en la BD .
cargar_desde_archivo (ruta)	ruta es string con ruta a archivo existente y legible. Cada línea con formato dd/mm/aaaa;temperatura .	Líneas válidas insertadas en la BD . Líneas con error se ignoran con aviso. Retorna cantidad cargada.

c. Resultados y pruebas

Se implementaron dos niveles de prueba: pruebas unitarias automáticas con unittest sobre el **ArbolAVL**, y una demo de uso manual sobre **TemperaturasDB**.

TEST 1: ArbolAVL (test_.py con unittest)

Se verifican **4 clases** de prueba:

- **TestArbolAVLInsertar:** árbol vacío, insertar uno y varios nodos, fecha duplicada actualiza temperatura, inorden ordenado, propiedad **AVL** tras inserción secuencial, decreciente y aleatoria.
- **TestArbolAVLBuscar:** buscar existente, inexistente y raíz.
- **TestArbolAVLBorrar:** borrar hoja, nodo con un hijo, nodo con dos hijos, borrar todos, borrar inexistente no cambia cantidad, propiedad **AVL** e inorden correctos tras borrado.
- **TestArbolAVLRango:** rango completo, parcial, un elemento, vacío, ordenado inorden, rango inverso devuelve vacío, **28** nodos aleatorios mantienen **AVL**.

```
PS C:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-TrabajoPractico_2> & C:\Users\franc\AppData\Local\Programs\Python\Python314\python.exe c:/Users/franc/OneDrive/Desktop/algoritmo/AyED2026c1-Acevedo-Tomiozzo-TrabajoPractico_2/Proyecto_2/tests/test_.py
.....
-----
Ran 27 tests in 0.024s

OK
```

TEST 2: TemperaturasDB (demo en Temperatura_DB.py)

Se verifica el flujo completo de la interfaz de alto nivel:

- Carga manual de **10** mediciones con **guardar_temperatura**.
- Consulta de **fecha** existente e inexistente con **devolver_temperatura**.
- Extremos en rango **01/06/2023** → **30/06/2023** con **temp_extremos_rango**.
- Listado ordenado en rango **10/06/2023** → **25/06/2023** con **devolver_temperaturas**.
- Borrado de medición y verificación con **devolver_temperatura**.
- Carga desde archivo **muestras.txt (120 registros)** con **cargar_desde_archivo**.

```
=====
TEMPERATURAS_DB - Demo de Kevin Kelvin
=====

Cargando mediciones manualmente...
Guardada: 01/06/2023 → 22.5 °C
Guardada: 05/06/2023 → 18.0 °C
Guardada: 10/06/2023 → -3.2 °C
Guardada: 15/06/2023 → 30.1 °C
Guardada: 20/06/2023 → 25.8 °C
Guardada: 25/06/2023 → 12.4 °C
Guardada: 30/06/2023 → 8.9 °C
Guardada: 04/07/2023 → 35.0 °C
Guardada: 10/07/2023 → -10.5 °C
Guardada: 20/07/2023 → 0.0 °C

Cantidad de muestras: 10

Consulta de fecha específica:
15/06/2023 → 30.1 °C
01/01/2000 → None

Extremos en rango 01/06/2023 → 30/06/2023:
Mínima: -3.2 °C | Máxima: 30.1 °C

Temperaturas en rango 10/06/2023 → 25/06/2023:
10/06/2023: -3.2 °C
15/06/2023: 30.1 °C
20/06/2023: 25.8 °C
25/06/2023: 12.4 °C

Borrando medición del 10/06/2023...
Muestras ahora: 9
10/06/2023 después de borrar: None

Cargando desde 'c:\Users\franc\OneDrive\Desktop\algoritmo\AyED2026c1-Acevedo-Tomiozzo-\TrabajoPractico_2\Proyecto_2\modules\..\data\muestras.txt'...
Carga completada: 120 mediciones cargadas, 0 errores.
Total de muestras: 120

=====
Fin de la demo.
```

3. ANÁLISIS DE RESULTADOS ESPECÍFICOS / DISCUSIÓN

La siguiente tabla resume la complejidad temporal de cada método. El análisis se basa en que el AVL es un árbol binario de búsqueda balanceado de altura $O(\log n)$, donde n es la cantidad de mediciones almacenadas. k denota la cantidad de elementos en el rango consultado.

Método	Complejidad	Justificación
guardar_temperatura(temp, fecha)	$O(\log n)$	Inserción en AVL: recorre la altura + hasta 2 rotaciones $O(1)$. El parseo de fecha es $O(1)$.



devolver_temperatura(fecha)	$O(\log n)$	Búsqueda en BST balanceado: recorre como máximo la altura del árbol.
borrar_temperatura(fecha)	$O(\log n)$	Eliminación en AVL : localizar en $O(\log n)$ + rebalanceo hacia la raíz en $O(\log n)$.
cantidad_muestras()	$O(1)$	Consulta directa del atributo _cantidad del ArbolAVL .
max_temp_rango(f1, f2)	$O(k + \log n)$	Obtiene los k nodos del rango en $O(k + \log n)$ con _rango_inorden , luego max() en $O(k)$.
min_temp_rango(f1, f2)	$O(k + \log n)$	Igual que max_temp_rango.min() sobre los k nodos del rango.
temp_extremos_rango(f1, f2)	$O(k + \log n)$	Una sola llamada a _rango_inorden + min() y max() en $O(k)$. Total $O(k + \log n)$.
devolver_temperaturas(f1, f2)	$O(k + \log n)$	_rango_inorden retorna los nodos ya ordenados (inorden = orden cronológico). Formatear k strings es $O(k)$.
cargar_desde_archivo(ruta)	$O(m \log m)$	m = líneas del archivo . Cada inserción es $O(\log m)$ en el peor caso cuando el árbol tiene m nodos .

El **AVL** es superior a una lista ordenada para este problema porque una lista requiere **$O(n)$** para insertar manteniendo el orden. Con el **AVL**, tanto las inserciones como las consultas se mantienen en **$O(\log n)$** , lo que es crítico cuando el conjunto de mediciones crece continuamente como en el caso de **Kevin**.

Para las consultas de rango, el término **$O(k + \log n)$** es óptimo: **$\log n$** para llegar al primer elemento del rango en el **árbol**, y **k** para recorrer los elementos dentro del rango. No es posible hacerlo mejor que **$O(k)$** ya que hay que visitar cada resultado.

La elección de **datetime** como clave del **AVL** fue fundamental. Permite comparar fechas directamente con **<** y **>**, lo que el **árbol** necesita para mantener el orden. Almacenar la fecha como string hubiera requerido parseo en cada comparación, degradando la performance.

El método **_rango_inorden** realiza poda implícita: no desciende por subárboles que no pueden contener elementos del rango. Esto es lo que garantiza **$O(k + \log n)$** en lugar de **$O(n)$** . Si la fecha del nodo actual ya supera **f2**, no hace falta explorar el subárbol derecho; si es menor que **f1**, no hace falta el izquierdo.



La separación entre **ArbolAVL** y **TemperaturasDB** permite reutilizar el árbol en otros contextos. **TemperaturasDB** solo maneja el parseo de fechas y el formato de salida, sin mezclar lógica de la estructura con lógica de la aplicación.

cargar_desde_archivo maneja errores línea a línea sin interrumpir la carga. Esto es importante en archivos reales donde puede haber líneas mal formateadas o valores fuera de rango. El método informa cuántas líneas se cargaron y cuántas tuvieron error.

4. DECLARACIÓN DE USO ÉTICO DE INTELIGENCIA ARTIFICIAL

Herramientas utilizadas: Claude (Anthropic), GitHub Copilot.

- **Consulta teórica:** se consultó para repasar las rotaciones del AVL y el análisis de complejidad de consultas por rango en árboles de búsqueda.
- **Asistencia en código:** se usó para revisar la lógica de `_rango_inorden` y la poda implícita, y para agregar pre/postcondiciones con validaciones a todos los métodos.
- **Redacción:** apoyo para mejorar la redacción y estructuración del informe.

Los integrantes del grupo declaramos que revisamos y validamos personalmente todo el código e información generada con IA. Entendemos los conceptos del informe y asumimos la responsabilidad académica por la integridad del contenido. El trabajo es producto de nuestro propio análisis.

5. TRABAJO EN EQUIPO

Estimación inicial: 7 horas en total: 2 horas de diseño del AVL, 3 horas de implementación y pruebas, 1 hora de análisis de complejidad y 1 hora para el informe.

Distribución de tareas:

- **Franco Tomiozzo:** implementación de **NodoAVL**, **ArbolAVL** (rotaciones, insertar, borrar, rango) y tests unitarios.
- **Franco Acevedo:** implementación de **TemperaturasDB**, carga desde archivo, demo de uso y redacción del informe.

Coordinación mediante WhatsApp y Discord, integración por commits en el repositorio GitHub. El punto más complejo fue implementar correctamente las cuatro rotaciones del AVL y verificar que el balanceo se mantenga tras borrados con sucesor inorden.

Tiempo real vs. estimado: aproximadamente 8 horas. La hora extra se fue en depurar los casos de borde del rango ($f1 > f2$, rango sin elementos, rango con un solo elemento) y en verificar el balanceo AVL con los tests unitarios.



6. CONCLUSIONES

Se implementó exitosamente la base de datos **TemperaturasDB** con todas las operaciones pedidas por la consigna, respaldada por un **Árbol AVL** balanceado. La estructura garantiza **$O(\log n)$** para operaciones individuales y **$O(k + \log n)$** para consultas de rango, lo que la hace adecuada para un conjunto de datos que crece continuamente.

Lo más valioso del trabajo fue entender la poda implícita en **_rango_inorden**: es lo que diferencia **$O(k + \log n)$** de **$O(n)$** en las consultas de rango, y es la razón principal por la que el **AVL** supera a otras estructuras para este problema.

La separación en capas (**NodoAVL** $\rightarrow$ **ArbolAVL** $\rightarrow$ **TemperaturasDB**) demostró su valor cuando se reutilizó el mismo **ArbolAVL** en otros problemas del TP sin modificaciones, lo que confirma el principio de diseñar estructuras genéricas desacopladas de su aplicación.

7. BIBLIOGRAFÍA

- Miller, B. & Ranum, D. Problem Solving with Algorithms and Data Structures using Python. Runestone Academy. <https://runestone.academy/>
- Cormen, T. et al. Introduction to Algorithms, 3ra ed. MIT Press, 2009.
- Repositorio del proyecto:
<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>

